

.NET 유니버스 2026

AI 에이전트 개발의 함정

속도가 만든 누더기 코드,
성당과 시장 사이에서 아키텍처를 다시 세우다

Speaker

김유신

Microsoft AI MVP | MCT | PreSales
Author | Speaker | Consultant

- 현) 메가존클라우드, Azure AI 프리세일즈
- 전) 클라우드 솔루션 아키텍트
- 전) 백엔드 개발자
- 전) 웹 개발자

IT 경력 25년 | AI 경력 4년



저서

- 『AI 에이전트 개발의 함정』 2026
- 『에이전틱 엔터프라이즈, 하네스 엔지니어링』 2026
- 『AI Agentic: 도구를 넘어 자율 협업의 시대로』 2026
- 『바이브 코딩: AI, 개발자, 그리고 소프트웨어 개발의 미래』 2025
- 『AI 도입, 왜 95%는 실패하는가?』 2025
- 『AI Agent 관찰·사고·행동의 모든 것』 2025
- 『코드 너머의 언어』 2024



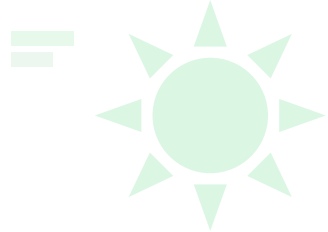


빠르게 만든 코드는 빠르게 무너집니다

AI는 개발 속도를 10배로 만들었지만, 설계 없는 속도는 고스란히 부채로 돌아옵니다.



목차



01

속도의 함정

에이전트가 코드를 대신 쓰기
시작하면서 개발 현장에서 달라진
것들

02

두 개의 청구서

누더기가 된 코드가 남긴 기술 부채,
그리고 갑자기 불어나는 토큰 요금

03

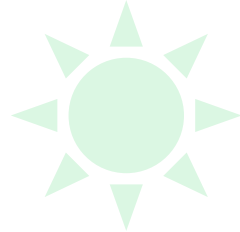
성당과 시장

무엇을 사람이 지키고 무엇을
에이전트에 맡길까 — 안전장치와 모델
나눠 쓰기로 되찾는 통제



속도의 함정

속도의 함정: 우리는 무엇을 위임했는가



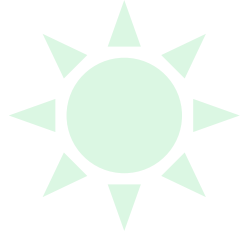
과거: 똑똑한 타자기

얼마 전까지 AI는 수동적이었습니다. 질문을 정확히 입력해야만 답을 내놓는 존재였죠.

오타를 잡아 주고 다음 단어를 추천하는 수준. 코드의 구조를 흔들 힘은 없었습니다.



속도의 함정: 우리는 무엇을 위임했는가



현재: 자율형 인턴

이제 AI는 목표만 주면 스스로 과정을 설계하고 끝까지 밀어붙입니다.

문제는 그 과정에서 만들어지는 구조를 아무도 승인하지 않았다는 점입니다.



선두주자: Codex & Claude Code



OpenAI Codex

큰 코드베이스를 빠르게 읽고 분석해 주는, 비서에 가까운 코딩 도구

Claude Code

복잡하게 얽힌 시스템 구조까지 파악해 스스로 문제를 해결하는 에이전트

이 인턴들은 코드를 쏟아냅니다. 문제는 그 코드를 끝까지 읽는 사람이 아무도 없다는 것입니다.



누더기가 되는 코드

이미 코드의 절반 이상은 우리가 쓰지 않습니다

41%

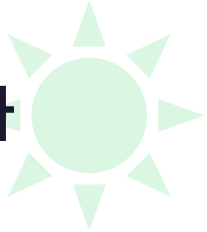
AI가 생성한 코드

그리고 남은 질문

미국 개발자의 92%가 AI 코딩 도구를 일상적으로
씁니다. 기술 의사결정자의 75%는 이미 심각한 기술
부채에 직면했다고 답했습니다.



원체스터 미스터리 하우스(스파케티 코드)는 이렇게 만들어집니다



맥락 없는 생성

에이전트는 저장소 전체를 읽지만
설계 의도까지 읽지는 못합니다.
매번 그럴듯한 새 경로를 하나 더
만들어 붙입니다

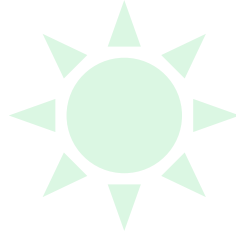
중복과 우회

이미 있는 함수를 찾는 대신 비슷한
함수를 또 만듭니다. 같은 규칙이
세 곳에서 서로 다르게 구현됩니다

검증 없는 병합

동작하니까 통과. 리뷰 없이 머지된
코드는 다음 세대 에이전트가
학습할 맥락이 됩니다

새로운 부채: 이해 부채(Comprehension Debt)



동작하는 코드

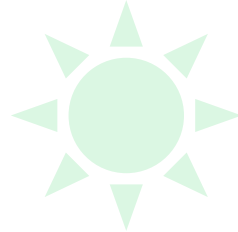
테스트는 통과하고 기능도 됩니다. 겉으로는 아무 문제가 없어 보입니다

아무도 모르는 코드

왜 그렇게 짜였는지 설명할 수 있는 사람이 없습니다.
장애가 터져야 비로소 읽기 시작합니다

기술 부채는 갚을 계획이라도 있습니다. 이해 부채는 갚아야 할 금액조차 모릅니다.

부채는 반드시 청구서로 돌아옵니다



속도

10배 빠른 생성



며칠 걸리던 기능이 몇 분으로
줄어듭니다. 여기까지는 모두가
박수를 칩니다

균열

전면 재설계



아키텍처 고민 없이 쌓인 코드는 결국
시스템을 통째로 다시 짜는 청구서로
돌아옵니다

규모

610억 근무일



17개국 3,000개 기업, 100억 줄
이상의 코드 분석이 추산한 글로벌
기술 부채의 무게입니다

빠르게 만드는 능력이 아니라, 빠르게 만든 것을 감당하는 능력이 경쟁력이 됩니다.



토큰이라는 두 번째 청구서

에이전트 비용은 왜 이렇게 폭발하는가

3,500x

최대 지출 폭증

스탠퍼드 연구결과

에이전트는 토큰을 어떻게 낭비 하는가?

1,000x ~ 3,500x

에이전틱 작업은 단일 토큰 챗봇 작업 대비 최대 3,500배의 토큰을 소모합니다.

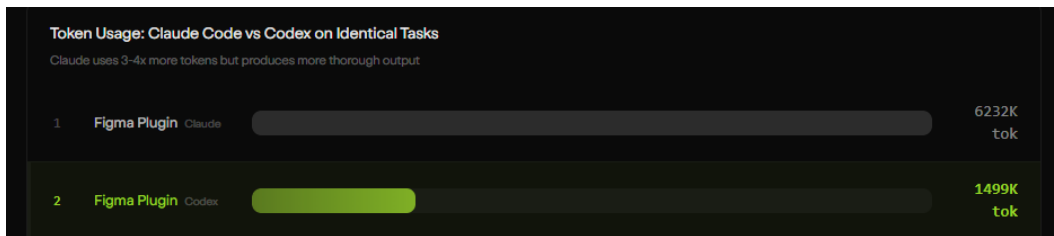
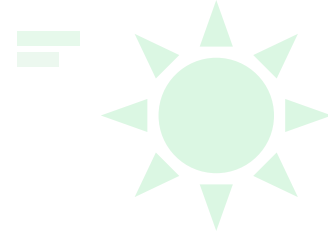
48.4K vs 30.4K

실제 GitHub 이슈 해결 시 평균 48,400개의 토큰이 소모되며, 이 중 30,400개는 에이전트의 도구 실행(Tool Output)을 다시 읽는 데 낭비됩니다.

30x Volatility

에이전트 궤적의 확률론적 특성으로 인해 동일한 작업에서도 최대 30배의 비용 변동성이 발생합니다. 모델은 자신의 비용성에 모델은 자신의 비용을 예측하지 못합니다.

실제 사례: Figma 플러그인 빌드

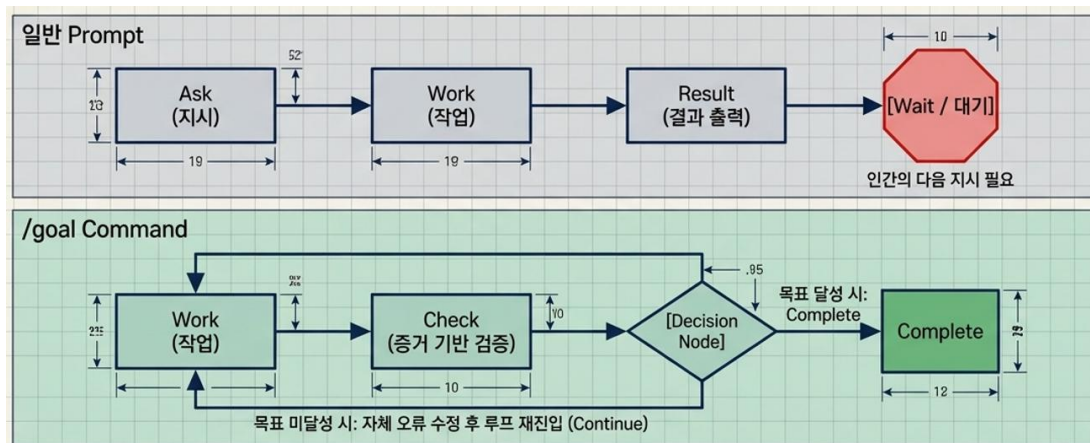


단일 기능을 구현하는 단 몇 분 만에
한 달 치 챗봇 사용량이 순식간에 증
발했습니다.

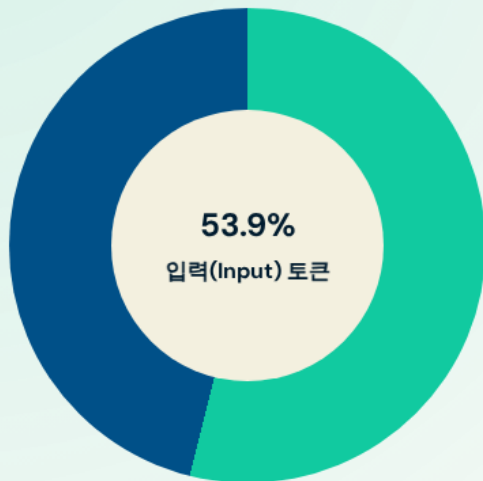
자율성: 이제는 '작업'이 아니라 '목표'를 줍니다

“다음 작업을 해” 가 아닌

“이 상태가 될 때까지 멈추지 마” /goal



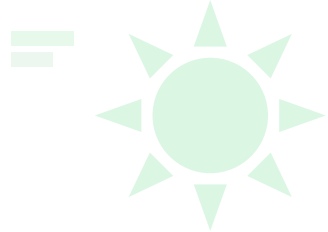
범인은 '다시 읽기' 비용



보이지 않는 지출의 실체

- ✓ 입력 토큰 비중: 53.9%
- ✓ AI가 새로운 것을 창작할 때보다, 기존 자료를 다시 읽어들이기 때 요금의 절반 이상이 발생합니다.
- ✓ 코드 한 줄을 고치기 위해 프로젝트 전체를 매번 다시 읽는 '비효율적 습관'이 원인입니다.

숨은 토큰 비용: Claude Code vs OpenCode



4.7×

질문 전에 쓰는 토큰

질문하기도 전에 Claude Code는
약 33,000토큰, OpenCode는 약
7,000토큰을 씁니다

54×

같은 내용 반복 전송

Claude Code는 같은 정보를 계속
다시 보내 캐시 비용이 최대 54배
더 듭니다

4.2×

나눌수록 늘어나는 비용

일을 2개로 나누면 토큰이 12만 →
51만으로 약 4배 늘어납니다

핵심: 비용의 절반은 '모델'이 아니라 '도구'에서 씁니다 — 눈에 안 보이는 토큰까지 관리하세요.

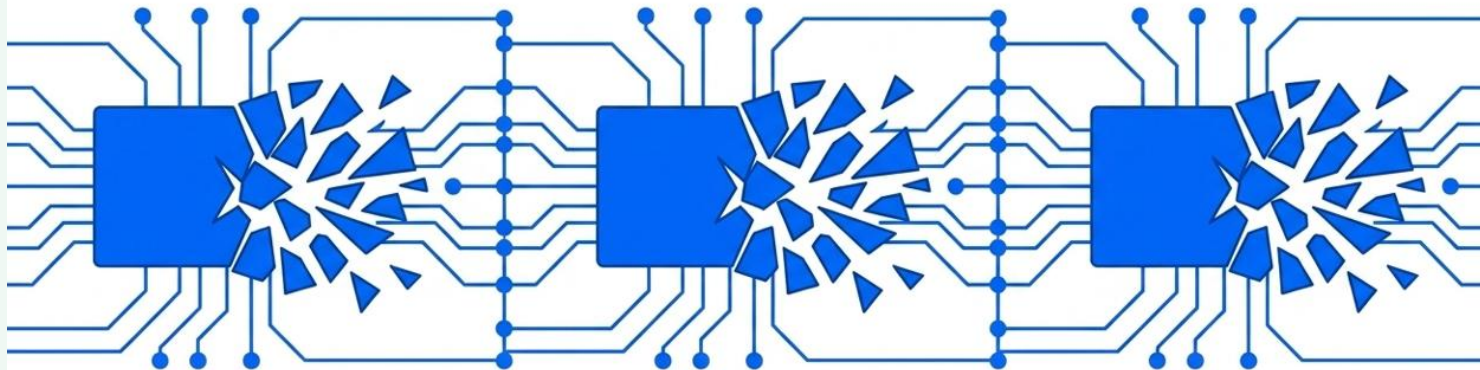


통제 실패의 3대 신호

캐시가 깨지고, 대화가 길어질수록 흐려지고, 너무 줄이면 손해 보는 것 — 통제가 무너지는 3가지 신호

캐시가 깨지는 순간 (Cache Invalidation)

정적 접두사(Prefix)의 단 한 글자만 바뀌어도 캐시 트리는 완전히 붕괴됩니다.



1. Model Flipping

세션 도중 모델 변경 (예: Opus ↔ Haiku). 모델별로 KV 캐시 구조가 다르기 때문에 즉각적인 캐시 붕괴 발생.

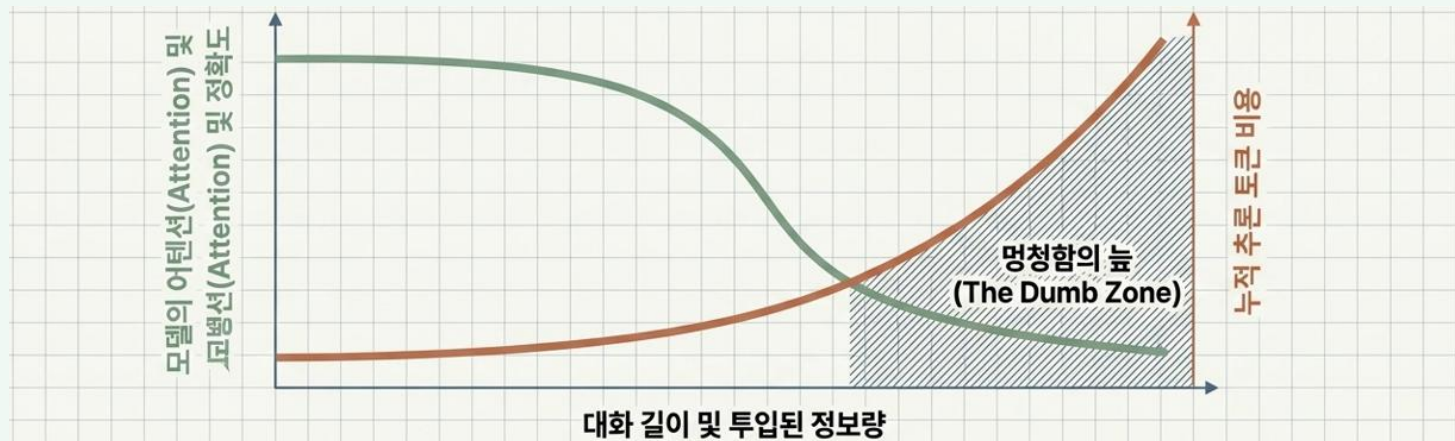
2. Tool Fluctuations

시스템 프롬프트에 내장된 MCP 서버 연결/해제 또는 동적 톨 스키마 변경.

3. Version Upgrades

백그라운드 CLI 업데이트로 인한 내부 시스템 프롬프트 변경.

대화가 길어질수록 흐려집니다 (Context Rot)



1. 엔트로피 폭발

과거의 실패 로그, API 응답, 불필요한 사유의 파편이 누적되며 핵심 제약 조건(보안규칙 등)에 대한 집중력이 상실됩니다.

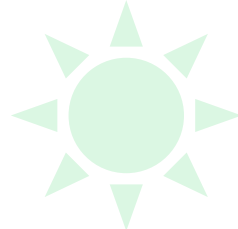
2. Dumb Zone 진입

지능이 저하된 상태에서 모델이 엉뚱한 라이브러리를 호출하거나 이전 실수를 무한 반복합니다.

3. 지수함수적 비용

단 한 번의 잘못된 루프로 수만 원~수십만 원의 비용이 즉각 청구됩니다. (압축/요약은 정보 손실을 유발해 또 다른 환각을 낳습니다).

너무 줄이면 오히려 손해 — 압축의 역설



개념

핵심 개념: 의미 밀도

전체 글자 수 중에서 '진짜 필요한 정보'가 차지하는 비율. 무조건 줄이는 게 아니라, 필요 없는 부분만 골라 없애는 게 핵심

1

예상 밖의 결과

코드를 17% 줄였더니(압축 방식 C), AI가 사라진 의미를 다시 짐작하느라 일을 더 많이 함. 그 결과 전체 비용이 67% 나 늘어남

2

남겨야 할 정보

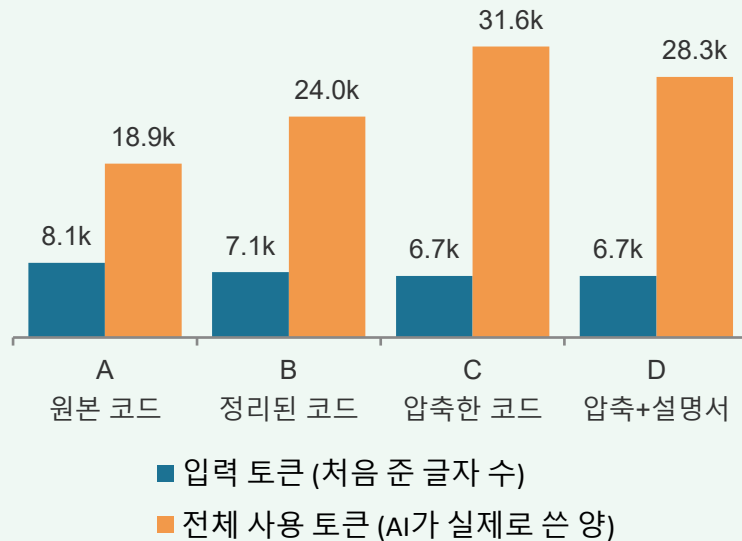
뜻이 담긴 변수 이름, 타입 표시, 자세한 설명, 친절한 에러 메시지. - AI가 헛갈리지 않도록 도와주는 배경지식

3

지워도 되는 정보

형식적으로 반복되는 코드, 중복된 접근 제어자, 자동으로 만들어지는 뼈대(틀) 코드는 지워도 괜찮음.

토큰 사용량 비교 (단위: 천 개)

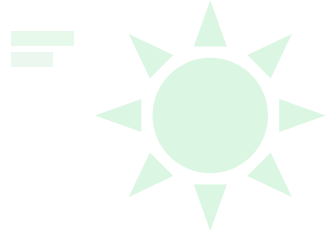


글자 수는 17% 줄었지만, **전체 비용은 67% 늘었어요**



성당인가, 시장인가

두 가지 아키텍처, 두 가지 통제 방식



성당 모델

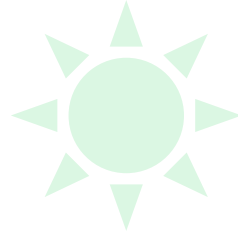
소수의 설계자가 구조를 쥐고 공개 전에 완성도를 책임집니다. 일관성은 높지만 속도가 느립니다

시장 모델

모두에게 열어 두고 빠르게 자주 배포합니다. 눈이 충분히 많으면 모든 버그는 알아진다는 원리입니다

에이전트 시대의 질문은 둘 중 무엇이냐가 아니라, 어디를 성당으로 잠그고 어디를 시장으로 열 것인가입니다.

에이전트에게 열어줄 곳과 잠글 곳



성당

경계와 계약



도메인 모델과 데이터 스키마, 공개
인터페이스는 사람이 설계하고 사람이
잠급니다

시장

구현과 실험



경계 안쪽의 구현과 테스트,
리팩터링은 에이전트에게 병렬로 활짝
열어 줍니다

심판

자동 검증



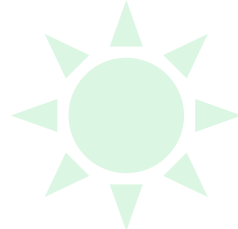
테스트와 정적 분석, 리뷰 게이트가
시장의 결과물을 성당의 기준으로
걸러냅니다

충분한 눈이 있으면 버그는 알아집니다. 다만 그 눈이 구조를 아는 눈일 때만 그렇습니다.



통제 아키텍처: 하네스(Harness)

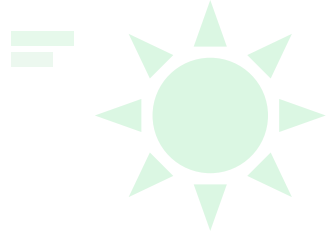
이제는 감이 아니라 시스템으로 통제합니다



	1단계: 프롬프트 엔지니어링	2단계: 컨텍스트 엔지니어링	3단계: 하네스 엔지니어링 (Harness)
제어 범위	메시지 레벨 (Message)	세션 레벨 (Session)	시스템 레벨 (System)
핵심 질문	어떻게 질문할 것인가?	무엇을 보여줄 것인가?	어떻게 통제하고 검증할 것인가?
컴퓨팅 비유	명령어 (Command)	작업 메모리 (RAM)	운영체제 (OS)
핵심 메커니즘	CoT, 역할 부여, Few-shot	RAG, 단/장기 메모리 주입	기계적 검증, 샌드박스, 랄프 루프
보안 모델	필터링 기반 (I/O)	권한 기반 (데이터 접근)	격리 기반 (물리적 제동/Kill Switch)
인간의 역할	지시자 (Operator)	정보 편집자 (Editor)	시스템 아키텍트 (Architect)

핵심 시사점: 프롬프트 최적화의 ROI 한계는 3% 미만입니다.
비약적 향상(28~47%)은 운영체제 레벨의 '하네스' 구축에서 나옵니다.

핵심 제어 기술: 효율의 극대화



프롬프트 캐싱

매번 똑같이 들어가는 프로젝트
설명을 재사용해 입력 비용을 최대
80% 절감

서킷 브레이커

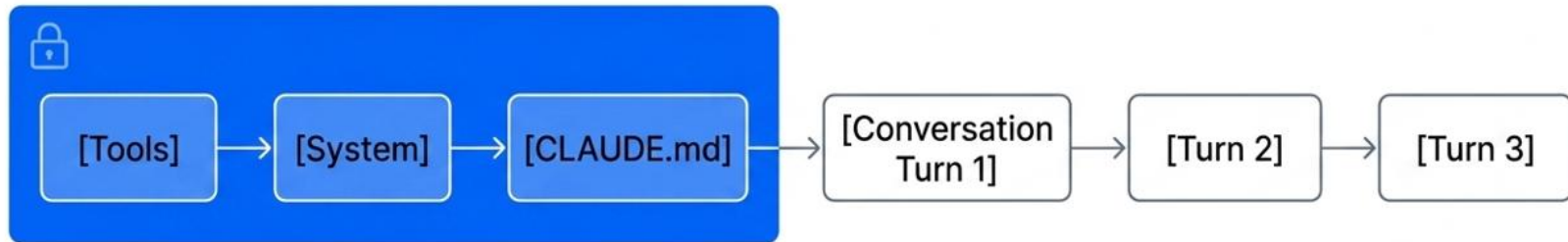
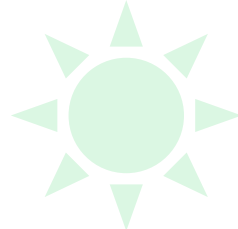
정해진 예산을 넘거나 이상한
반복이 감지되면 호출 자체를 끊어
버립니다

랄프 루프 (Ralph Loop)

단계마다 쌓인 에러 로그를 지워
AI가 헛갈리지 않게 만드는 방법



앞부분을 고정해 재사용하는 프롬프트 캐싱



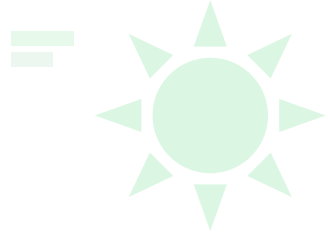
정적 접두사 (Static Prefix) /
Cached State (80% Cost Reduction)

동적 꼬리 (Dynamic Tail) / 재계산 영역

매 턴마다 전체 텍스트를 재계산(KV Cache)하는 대신,
변하지 않는 '정적 접두사(Static Prefix)'를 재사용하여
입력 토큰 처리 비용 및 첫 바이트 도달 시간(TTFB)을
최대 80% 감소시킵니다.

Design Rule: 재사용 가능한 정적 컨텍스트(코드베이스
요약, 톨 정의)는 반드시 프롬프트의 가장 앞단(Base
Camp)에 배치해야 합니다.

프롬프트 캐싱은 이렇게 조용히 깨집니다



1

도구 목록 변경

도구 목록은 프롬프트 맨 앞에 있습니다. 하나만 더하거나 지워도 그 뒤가 전부 새것으로 취급돼 캐시가 날아갑니다

2

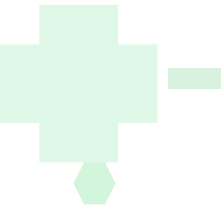
5분의 공백

캐시는 기본 5분이면 사라집니다. 테스트 돌리고 커피 마시고 와서 "계속" 한 마디를 보내면 10만 토큰 전체가 정가로 다시 청구됩니다

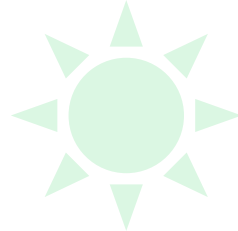
3

착한 정리의 배신

비용을 아끼려 오래된 기록을 지우면 그 뒤가 전부 캐시 미스. 아낀 토큰보다 다시 계산하는 비용이 더 큼니다. 지우지 말고 뒤에 붙이세요



캐시가 한 번 깨지면 그 세션에서 가장 비쌉니다



캐시 적중 — 입력 요금의 10%

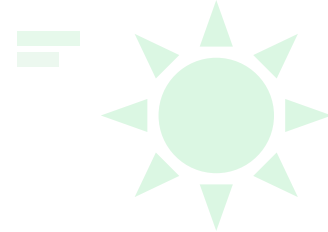
캐시가 살아 있으면 10만 토큰 대부분이 할인 가격으로 처리됩니다. 세션이 길어질수록 이득도 함께 커집니다

캐시 실패 — 정가 + 저장 비용

한 번 어긋나면 10만 토큰 전체를 100% 가격으로 다시 내고 캐시 저장 비용까지 붙습니다

긴 세션에서는 캐시 적중률이 곧 비용입니다. 요금이 이상하게 높다면 모델보다 적중률부터 확인하세요.

서킷 브레이커 (Circuit Breaker)



1

예산 임계치 설정

시간당 50달러 등 지출 가능한 상
한선을 미리 설정

2

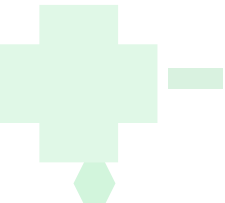
실시간 추적

통신 중인 토큰 요금을 1초 단위로
감시 및 합산

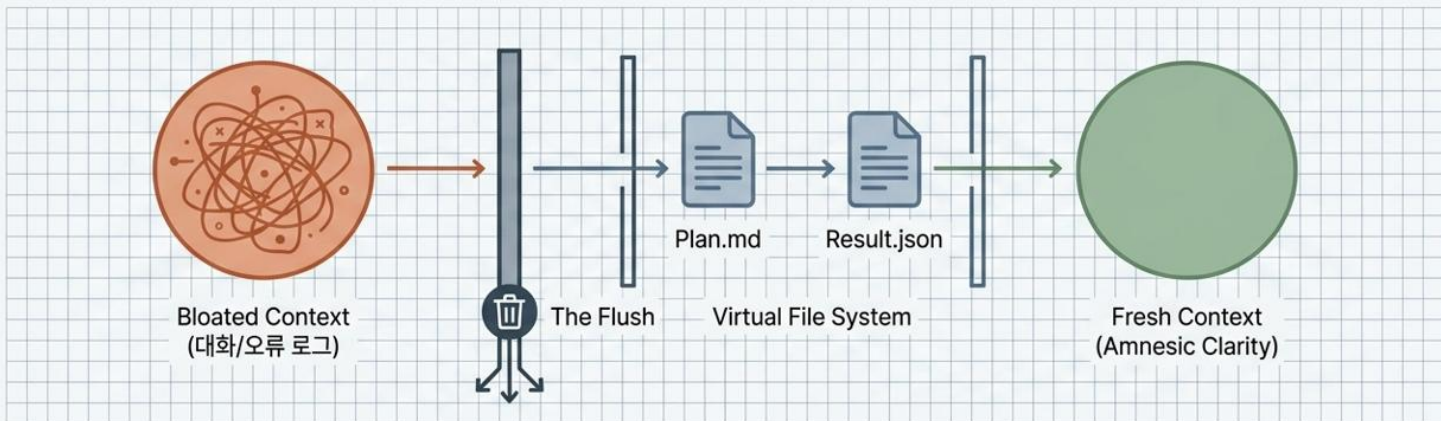
3

물리적 차단

한도를 넘기는 즉시 실행 중인
명령을 강제로 종료합니다



랄프 루프(Ralph Loop): 의도적 망각



1. 의도적 망각 (Intentional Amnesia)

단위 작업(Task)이 종료되는 순간, 에이전트의 이전 대화 로그, 실패한 시도의 흔적, 오류 메시지를 강제로 100% 소거(Flush)합니다.

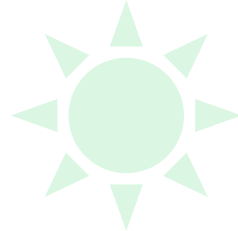
2. VFS (가상 파일 시스템) 인수인계

모델의 내부 메모리에 의존하지 않습니다. 오직 철저히 구조화된 두 개의 텍스트 파일(Plan.md와 Result.json)만을 하네스가 외부 상태로 저장하여 다음 루프에 주입합니다.

3. 명징함의 유지 (Amnesic Clarity)

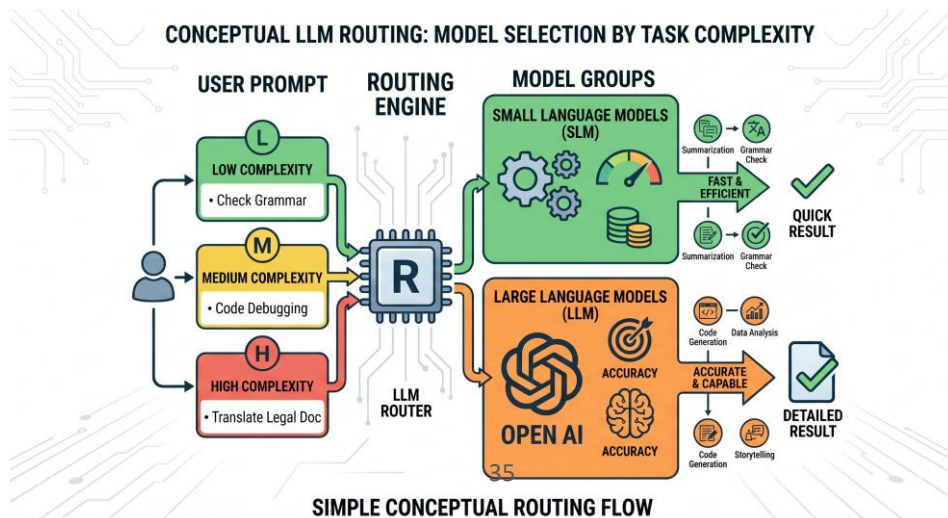
에이전트는 매번 텅 빈 깨끗한 컨텍스트 창에서 오직 '당면한 단일 목표'에만 뇌의 100%를 집중합니다. 멍청함의 늪(Dumb Zone) 진입 확률을 물리적으로 0에 수렴시킵니다.

하이브리드 교차 투입 전략



업무 난이도에 따른 LLM 모델 사용

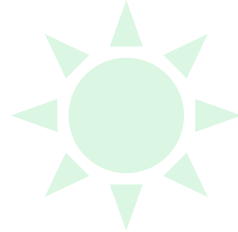
전문가는 하나의 비싼 모델만 고집하지 않습니다. 단순하고 빠른 일은 가성비 좋은 모델에게, 복잡한 설계와 최종 검토는 고성능 모델에게 나누어 배정하여 지출 효율(ROI)을 극대화합니다.





가격 붕괴, 그리고 모델 오케스트레이션

2026년 7월 30일, 가격이 저렴해진 날



GPT-5.6 Luna

80% 인하



출시 3주 만의 조정. 서비스 운영 비용
20% 절감과 토큰 생성 효율 15%
개선이 근거로 제시됐습니다

GPT-5.6 Terra

20% 인하



모델이 스스로 최적화한 인프라
비용이 곧바로 가격표에 반영된
사례입니다

GPT-5.6 Sol

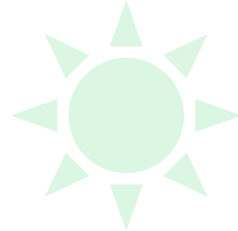
가격 동결



플래그십은 인하 대상에서 제외. 입력
\$5 / 출력 \$30 을 그대로 유지합니다

OpenRouter 는 여기에 50% 추가 할인을 얹어 Luna 를 입력 \$0.10 / 출력 \$0.60 까지 내렸습니다.

AI 는 싸지면서 동시에 비싸지고 있습니다



약 89배

Opus 4.8 출력 \$25 대 DeepSeek V4 Flash 출력 \$0.28. 같은 한 줄의 답을 받는 비용 차이입니다

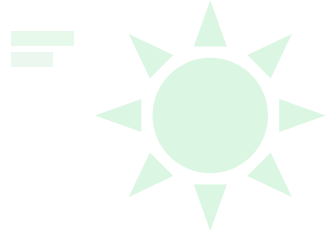
약 83배

Fable 5 출력 \$50 대 OpenRouter Luna 출력 \$0.60. 경량 라인은 무너지고 있습니다

역설의 정체

반대로 메인 모델은 오히려 오릅니다. Fable 5 는 입력 \$10 / 출력 \$50 으로 Opus 4.8 의 정확히 2배입니다

추가 결제 없이 시작하는 오케스트레이션



1

보조 모델부터 만든다

이미 구독 중이라면 추가 결제 없이
기존 할당량만으로 경량 모델 보조
작업자를 돌릴 수 있습니다. 가장
쉬운 출발점입니다

2

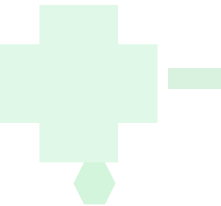
호출 기준을 문서화한다

독립적인 큰 작업은 보조 모델에
병렬로 맡기고, 각 작업의 파일
범위와 기대 결과를 함께
전달하도록 규칙에 적어 둡니다

3

역할을 고정한다

메인 모델은 설계와 최종 검토,
보조 모델은 조사와 구현. 반드시
보조 역할로 두고 추론 강도는 항상
최대로





코드를 만들기 전에 구조를 먼저 세우십시오

감사합니다.